Supplementary Material: Mitigating False Completion in Vision-Language-Action Models

Anonymous Authors

This appendix provides comprehensive supplementary material to support the methodology, analysis, and findings presented in the main paper. It includes implementation details covering model architectures and training configurations, along with additional experimental results and analyses. The appendix is organized as follows.

- **Section I** provides further analysis of state-dominant bias under imitation learning and explains the resulting design motivation for ReViP.
- **Section II** details the task-stage observer and its reasoning process, action generation, training configuration, asynchronous inference, and ReViP-Flash implementation.
- **Section III** specifies benchmark construction, FCSR validity criteria, perturbation protocols, and simulation and real-world evaluation settings.
- **Section IV** reports additional results on RoboTwin 2.0, sensitivity analyses, and evaluations of robustness to observer errors.
- **Section V** presents additional simulation and real-world rollouts, together with visualizations of task-relevant visual responses.

Video Demo

To provide an intuitive illustration of our model's performance and comparative results, we include a video demonstration in the supplementary materials, titled *Video_Demo.mp4*.

I. Additional Analysis of False Completion

This section provides a formal interpretation of the state-dominant bias suggested by the diagnostics in the main paper. We examine how correlations in successful demonstrations can leave visual verification of task progress under-constrained, and explain how this analysis motivates the two complementary mechanisms of ReViP.

A. Training Distribution and State-Dominant Bias under Imitation Learning

We provide a formal interpretation of the state-dominant bias suggested by the diagnostics in the main paper, focusing on how the training distribution and imitation learning objective shape implicit task-progress grounding.

Training-Distribution Correlation. Let $\mathcal{D}_{\mathrm{std}} = \{(I_t, S_t, \ell, A_t)\}$ denote the standard distribution of successful manipulation trajectories, where I_t, S_t, ℓ, and A_t denote the visual observation, proprioceptive state, task instruction, and expert action, respectively. For a fixed task instruction ℓ and a relevant task stage k, let $G_k(I_t, \ell) \in 0, 1$ indicate whether visual evidence supports the condition required for advancing beyond that stage. Let $\mathcal{S}_k^+(\ell)$ denote the proprioceptive-state region typically associated with such advancement. Within the portion of successful demonstrations where this condition is relevant, we consider the case in which these cues are strongly correlated:

$$\Pr_{\mathcal{D}_{\mathrm{std}}}\left[G_k(I_t, \ell) = 1 \mid S_t \in \mathcal{S}_k^+\right] \approx 1. \tag{1}$$

Thus, under this assumption, proprioceptive transitions provide a highly predictive cue of the policy's current task stage for stage advancement, even without independently verifying the corresponding visual condition.

Behavior-Cloning Consequence. Behavior cloning optimizes expert action reproduction:

$$\theta^\star = \arg\min_\theta \mathbb{E}_{(I,S,\ell,A)\sim\mathcal{D}_{\mathrm{std}}} \mathcal{L}_{\mathrm{BC}}\big(\pi_\theta(I, S, \ell), A\big). \tag{2}$$

where $\mathcal{L}_{\mathrm{BC}}$ denotes the action-imitation loss of the underlying imitated-based policy. This objective can encourage visual responsiveness when it is needed to reproduce expert actions. However, this objective does not explicitly require the policy to verify that its presumed task progress remains supported by current visual evidence, *i.e*, action supervision alone may not distinguish policies that verify visual stage conditions from those that rely primarily on correlated state cues. In particular, it imposes no explicit consistency constraint of the form $\hat{p}_t = 1 \implies G_k(I_t, \ell) = 1$, where, as in the main paper, $\hat{p}_t = 1$ conceptually denotes behavior consistent with treating the relevant stage condition as satisfied. It is neither an explicit policy output nor a measured internal state. Consequently, execution-state cues can serve as a proxy for task progress, while the policy's response to conflicting visual evidence remains weakly constrained by the training data. This ambiguity can favor the *state-dominant bias in progress estimation* suggested by our empirical diagnostics.

False completion under visual-progress conflicts. At deployment, unexpected disturbances can produce observations that violate the training-time correlation:

$$S_t \in \mathcal{S}_k^+(\ell), \qquad G_k(I_t, \ell) = 0. \tag{3}$$

The execution state still resembles states associated with task stage advancement, but current visual evidence no longer supports the required condition. For example, after an object drops, the robot may remain on its transport trajectory even though the object is no longer held. A visually responsive policy can revise its actions in response to this discrepancy.

A state-dominant policy may instead continue to behave as if the condition were satisfied, corresponding to $\hat{p}_t = 1$ despite $G_k(I_t, \ell) = 0$. False completion arises when the policy advances or terminates execution on this unsupported assumption. Thus, receiving fresh observations alone does not ensure correction: the conflicting visual evidence must influence subsequent action generation.

B. ReViP for False Completion

The above analysis directly motivates explicitly grounding task progress in visual evidence and using this evidence to condition how the policy processes its inputs. ReViP implements this principle through *Visual-Progress Grounding* and *Stage-Aware Attention Rebalancing*. These mechanisms retain the underlying action-training objective while introducing a direct pathway from visual stage verification to action generation.

Formulation of ReViP. Let P_t denote the vision-language context produced by the base VLA. ReViP first obtains visual-progress cues through task-stage reasoning:

$$z_t = \mathrm{TSO}(I_t, \ell), \tag{4}$$

where TSO verifies task-stage conditions against the current visual observation and encodes the resulting evidence into z_t. These cues are distinct from $\hat{p}_t$: the latter describes the base policy's presumed progress conceptually, whereas z_t is an explicit representation used to condition the policy. We express the resulting action model $\pi_{\theta,\phi}^{\mathrm{ReViP}}$ as:

$$\begin{aligned} \widetilde{P}_t &= \mathcal{T}\phi(P_t; z_t), \\ \delta_t &= \mathcal{R}\phi(\widetilde{P}_t, S_t; z_t), \\ \pi_{\theta,\phi}^{\mathrm{ReViP}}(A_t \mid I_t, S_t, \ell) &= \pi_\theta(A_t \mid \widetilde{P}_t, S_t; \delta_t). \end{aligned} \tag{5}$$

where θ denotes the base-policy and ϕ denotes the grounding and rebalancing modules. $\mathcal{T}\phi$ implements FiLM-based context modulation, while $\mathcal{R}\phi$ produces the calibrated attention modulation δ_t. Here, π_θ denotes the model component conditioned on $\widetilde{P}_t$ and S_t, with δ_t applied to its attention logits.

This formulation makes visual stage evidence a conditioning signal for both the policy representation and the relative influence of multimodal inputs. Visual-Progress Grounding incorporates z_t into P_t, allowing the context supplied to the action expert to reflect visually supported task progress. Stage-Aware Attention Rebalancing further uses these cues and execution context to adjust how the action expert attends to visual, language, state, and action tokens. Thus, ReViP learns not only to incorporate task-stage evidence into policy representations but also to regulate its influence during action generation.

Design Motivation for Stage-Aware Attention Rebalancing. An intuitive strategy is to uniformly amplify visual information and suppress state information. However, such fixed reweighting overlooks two properties of visual-progress conflicts. First, proprioceptive cues can become misleading while remaining useful for reliable manipulation. Their influence should therefore depend on both current visual stage evidence and execution context. Second, evidence that invalidates a stage condition is often localized around manipulated objects, the robot body, and their spatial relations, rather than distributed uniformly across image patches.

Effective rebalancing must therefore determine *which* evidence to promote or suppress, *how strongly* to modulate attention, and *where* to allocate visual modulation. Stage-Aware Attention Rebalancing addresses these requirements by learning modulation signals conditioned on visual-progress cues and execution state, with group-wise dual calibration controlling their strength and token-wise allocation. This enables selective emphasis on stage-relevant visual evidence while retaining useful execution cues.

Closed-Loop Deployment. When a disturbance invalidates a stage condition, the robot's execution state may still resemble states associated with successful advancement. ReViP allows new visual evidence to affect action generation through updated cues z_t, which reshape both the context $\widetilde{P}_t$ and attention modulation δ_t. The policy can thus revise subsequent actions without requiring proprioceptive state alone to signal that the presumed progress is no longer valid.

As execution proceeds, newly available observations refresh the cues and their influence on the policy, closing the loop between visual stage verification and action generation. ReViP therefore preserves proprioceptive information while introducing an explicit pathway for visually grounded action revision. This design does not enforce a hard stage-consistency constraint; its effectiveness depends on the quality of the visual-progress cues and how the learned modulation translates them into revised actions.

II. Additional Implementation Details

This section describes how ReViP is implemented and trained, including task-stage cue extraction, action generation, and baseline adaptations. We also detail asynchronous inference and ReViP-Flash distillation for efficient deployment.

A. Model Details

Task-Stage Observer. The default setting of Task-Stage Observer (TSO) is built upon the Qwen 2.5 VL model [1] and is designed to extract structured, observation-grounded task cues that complement the robotic policy. It receives the task instruction, a task-goal prompt, and a third-person image covering the workspace, providing visual context for verifying task-stage conditions under perturbations.

Given the task instruction and current observation, TSO follows the structured reasoning protocol in Table I to extract task-relevant visual evidence, verify the currently valid task stage, and infer the immediate execution intention. This protocol guides the model to enumerate visual evidence, specify stage-wise completion conditions, and infer an explicit task stage description that is aligned with the ongoing trajectory. Let $f_t \in \mathbb{R}^{d_{\mathrm{TSO}}}$ denote the pooled last-layer hidden feature produced of TSO. A lightweight projection maps it to the visual-progress cue

$$z_t = \mathrm{Proj}(f_t) \in \mathbb{R}^d, \tag{6}$$

where $d = 2048$ in our implementation. The resulting cue conditions FiLM-based context modulation in Visual-Progress

TABLE I
INSTRUCTION DETAILS FOR THE TASK-STAGE OBSERVER.

Component	Content
System Prompt	You are an expert robot task assistant, integrated with a robot arm capable of executing physical actions in the real world. Based on the task goal and observation, provide the additional task-stage information to the robot in a structured and precise way.
Reasoning Rules	• *Task Decomposition:* DECOMPOSE the instruction into ordered stage-level objectives and specify the visual completion condition of each stage. • *Visual-Evidence Extraction:* EXTRACT only task-relevant visual evidence, including target-object status, robot–object relations, and task-relevant spatial relations. • *Stage Verification:* DETERMINE the currently valid stage by comparing the visible evidence with the stage completion conditions. Do not assume monotonic progress; revert to an earlier stage if a previously satisfied condition is visually invalidated. • *Intention Inference:* INFER the immediate execution intention according to the verified stage, including corrective intentions when necessary.
Input Fields	Task Goal: `{task_goal}` Observation: `{Third person view image}`

Output Format	`task_stages`: [`stage_1`, ..., `stage_K`]
	`visual_evidence`: [`evidence_1`, ...]
	`verified_stage`: `current_valid_stage`
	`immediate_intention`: `next_execution_objective`

Grounding and attention modulation in Stage-Aware Attention Rebalancing, as defined in the main paper.

B. Action Generation

Following the flow-matching objective, the action is trained to predict a conditional velocity field $u_\theta\big(.\ ,\ .\big|\ z_t\big)$ via flow matching on the progress-aware vision–language representation $\widetilde{P}_t(z_t)$ and calibrated attention modulation δ_t. For a ground-truth action chunk A_t and random noise ε, we sample $\tau \sim \mathcal{U}(0,1)$ and construct a noisy action along the straight-line interpolation:

$$x_t^\tau \ = \ (1-\tau)A_t + \tau\,\varepsilon, \tag{7}$$

The target velocity (*i.e*, the time derivative of this path) is:

$$u_t^\tau = \frac{dx_t^\tau}{d\tau} = \varepsilon - A_t, \tag{8}$$

Above all, the training objective regresses the predicted velocity field toward this target:

$$\mathcal{L}_{\text{action}} = \|v_\theta(x_t^\tau, \tau \mid H_t) - u_t^\tau\|_2^2 . \tag{9}$$

During inference, actions are generated by integrating the predicted velocity field from noise at $\tau = 1$ toward $\tau = 0$. Thus, ReViP preserves the original action space and flow-matching objective while modifying the representations and attention used for action generation. For example, after an object drop, the updated z_t first re-grounds the policy representation through FiLM and guides Stage-Aware Attention Rebalancing, enabling the policy to return to and regrasp the object rather than continuing toward the goal, reducing false completion.

C. Training Details

Training Setup. We adopt π_0 [2] as the default backbone model with full parameter fine-tuning and set the action chunk size to $K = 50$. The model is trained for 60k steps with a batch size of 32. The learning rate follows a cosine decay schedule with 3,000 warmup steps, increasing to a peak of 2.0×10^{-5} and then decaying to a final learning rate of 2.0×10^{-6}. We use the AdamW optimizer with gradient clipping set to 1.0 and apply an exponential moving average (EMA) with a decay value of 0.999.

Task-Stage Annotation. To avoid training latency caused by online task-stage extraction, we preprocess the entire dataset using the Task-Stage Observer before training. For every action step, we extract the corresponding task-stage intention and store it as part of the training annotations. During training, the task-stage cues are injected into ReViP to provide explicit task-stage guidance, enabling the model to more effectively rebalance visual and proprioceptive features. At inference time, task-stage intention is computed online by the Task-Stage Observer and fed into ReViP to guide ongoing action generation.

Implementation of Agentic Baselines. We adapt VoLoA-gent [3] and HarnessVLA [4] from their official open-source implementations for evaluation on our False-Completion Benchmark Suite. For both methods, we replace the underlying VLA policy with the same π_0 backbone used by ReViP, while retaining their high-level planning, feedback, and recovery mechanisms. The adapted systems are evaluated under the same task configurations, perturbation protocols, and evaluation criteria as ReViP. This shared-backbone setting allows us to compare external feedback and recovery with visual-progress rebalancing while controlling for differences in the underlying manipulation policy. These results therefore correspond to our π_0-based adaptations for fair comparison.

D. Additional Analysis of Deployment Efficiency

Asynchronous Inference Pipeline of ReViP. As illustrated in Figure 1, TSO performs task-stage reasoning once per action chunk and runs asynchronously with robot execution. The resulting visual-progress cue is consumed by FiLM-based

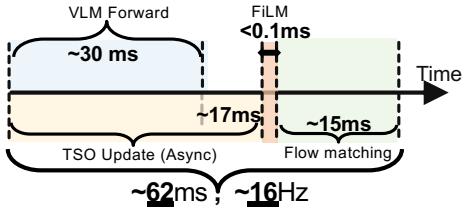


Fig. 1. **Inference pipeline of ReViP.** TSO donates the task-stage observer. The task-stage reasoning is performed once per action chunk, while the lightweight policy-side modulation introduces negligible additional latency.

context modulation and Stage-Aware Attention Rebalancing before the next action chunk is generated. This pipeline overlaps observer inference with action execution, hiding part of the external-observer latency.

Comparison with Agentic Recovery Online agentic recovery introduces runtime costs beyond those of the underlying manipulation policy. For example, according to the official results [4], HarnessVLA reports mean planner durations of 344.17–529.23,s on LIBERO-PRO [5] under its evaluated online Codex configurations.

ReViP instead incorporates visual stage evidence into policy representations and action-generating attention, allowing corrective actions without invoking an external agent-level planner for each correction. In our evaluation, ReViP supports corrective action generation at approximately 15,Hz, with a policy inference latency of 62,ms per action chunk and only 17,ms of additional policy-side overhead. External TSO inference runs asynchronously with robot execution. ReViP-Flash further eliminates external VLM inference by distilling TSO into a lightweight in-policy observer, while retaining the performance of ReViP.

These results highlight the low policy-side overhead of integrating visual-progress correction into action generation. The cited planner durations and our policy inference measurements cover different components and evaluation settings and therefore do not establish a direct speedup ratio.

E. ReViP-Flash Implementation

ReViP-Flash replaces the external TSO with an event-aware in-policy Q-Former at deployment. The semantic stream uses VLM hidden states, while the temporal stream captures differences between consecutive visual-encoder features. Both reuse representations produced by the VLA backbone without introducing an additional visual encoder. The previous visual features are cached at each action-chunk replanning step.

For training, we first run the external TSO offline and cache its visual-progress cues z_t as teacher targets. The Q-Former predicts $\hat{z}_t$ and is trained using the directional cue-alignment loss $\mathcal{L}\mathrm{sim}$, cue-scale alignment loss $\mathcal{L}\mathrm{scale}$, and downstream action loss $\mathcal{L}_{\mathrm{action}}$ defined in the main paper.

During distillation, the VLA backbone, FiLM-based context modulation, and Stage-Aware Attention Rebalancing remain frozen. Only the event-aware Q-Former and its output projection are optimized. The frozen downstream modules remain differentiable with respect to $\hat{z}_t$, allowing the action loss to backpropagate through context modulation and attention rebalancing. This trains the in-policy observer to produce cues that support action generation, in addition to matching the teacher cues.

At deployment, $\hat{z}_t$ directly replaces the external TSO cue z_t, while the downstream ReViP pipeline remains unchanged.

III. Additional Experimental Details

This section specifies how visual-progress conflicts are constructed and evaluated, including task-specific FCSR validity criteria and perturbation protocols. It also describes the additional simulation benchmarks and real-world settings used to assess performance beyond the controlled conflicts.

A. False Completion Benchmark Suite Details

False completion arises when a visual change invalidates the presumed task progress, while the policy continues or terminates according to the outdated progress assumption. To study state-dominant bias and visually apparent false completions in a controlled manner, we construct a False Completion Benchmark Suite on top of LIBERO that injects targeted perturbations and probes complementary aspects of the phenomenon.

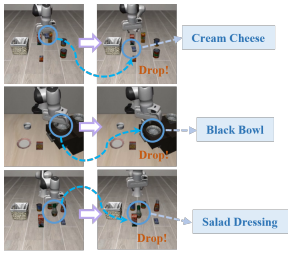
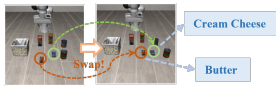
We consider three perturbation sources that commonly induce false completion: *Object Drop*, *Distractor Swap*, and *Object Relayout*. Together, these perturbations form a suite of **8 manipulation tasks** that constitute the proposed False Completion Benchmark. The instruction remains fixed, and only the environment state is perturbed either at the start of the episode or during execution. This design breaks demonstration-specific execution and spatial correlations, requiring policies to rely on current visual evidence for progress verification and action revision. An overview of all tasks, including representative visualizations and the corresponding language instructions, is provided in Table II. Below, we detail the three perturbation families.

Object Drop. This setting tests recovery from unexpected displacement and exposes false completion most directly. We instantiate three variants that differ in object scale and contact context, which in total yield five tasks:

- *Small Object Drop:* a piece of cheese and a block of butter are grasped and then forcibly dropped onto the table, requiring the policy to visually re-detect the fallen small object and regrasp it.
- *Large Object Drop:* a bottle of salad dressing and a carton of orange juice are lifted and then dropped, which introduces longer trajectories and larger motion amplitude compared with small object cases.
- *Bowl Drop in Cabinet:* a bowl is dropped inside a cabinet with occlusion and contact constraints, and the policy needs to visually localize the fallen bowl and complete the task under clutter and limited visibility.

Distractor Swap. This setting evaluates whether the policy detects that two visually similar objects have exchanged positions rather than replaying a proprioception-driven trajectory. The initial poses of the target and a visually similar distractor are swapped while the instruction remains fixed. Concretely, the initial locations of cheese and butter are exchanged while the

TABLE II
OVERVIEW OF THE FALSE-COMPLETION BENCHMARK SUITE. THE SUITE COMPRISES EIGHT MANIPULATION TASKS ORGANIZED INTO THREE CONTROLLED PERTURBATION FAMILIES: *Object Drop*, *Distractor Swap*, AND *Relayout*, EACH TARGETING A DISTINCT SOURCE OF VISUAL-PROGRESS CONFLICT. REPRESENTATIVE VISUALIZATIONS AND THE CORRESPONDING LANGUAGE INSTRUCTIONS ARE PROVIDED FOR EACH FAMILY. TARGET OBJECTS AND GOAL REGIONS ARE HIGHLIGHTED IN PURPLE AND TEAL, RESPECTIVELY.

Type	Instruction	Visualization Example
Object Drop	❶ Pick up the butter and place it in the basket. ❷ Pick up the cream cheese and place it in the basket. ❸ Pick up the salad dressing and place it in the basket. ❹ Pick up the orange juice and place it in the basket. ❺ Pick up the black bowl in the top drawer of the wooden cabinet and place it on the plate.	
Distractor Swap	❻ Pick up the butter and place it in the basket. ❼ Pick up the cream cheese and place it in the basket.	
Object Relayout	❽ Pick up the black bowl next to the plate and place it on the plate.	

instruction remains unchanged, and the policy must select and manipulate the linguistically specified target. This produces two tasks that differ only in which object is specified as the linguistic target.

Object Relayout. This setting breaks demonstration-specific spatial priors while preserving the linguistic target. The target object and its goal region are jointly relocated to new feasible positions within the workspace. The policy is therefore required to update its plan based on the current visual configuration rather than relying on stale internal state or memorized spatial layouts. In our benchmark, this gives one relayout task that relocates both the target and its goal surface.

Overall, the three perturbation families jointly emphasize visual grounding, continuous progress monitoring, and effective replanning under distribution shifts that frequently result in false completions. The suite provides a focused and diagnostic evaluation framework for VLA models, isolating failure modes that arise when policies overly trust internal state progression in the presence of visually apparent disturbances.

B. Task-Specific FCSR Validity Criteria

The main paper defines FCSR over the set of trials that validly reach the stage at which the policy must respond to the intended visual-progress conflict. Here, we specify the task-specific validity criteria used to construct this set.

- *Object Drop.* A trial is valid only after the target object has been successfully grasped and lifted and the deterministic drop trigger has been executed. Failures occurring before the drop are excluded. Once the drop is triggered, the trial remains in the FCSR denominator and is successful only if the policy completes the instructed task within the step budget.
- *Distractor Swap.* The target–distractor swap is applied before the first policy observation. A trial is valid once the environment is successfully initialized with the prescribed swapped configuration. All subsequent policy outcomes are included in FCSR, including selecting the wrong object, approaching its original location, failing to grasp, or terminating prematurely. Success requires completing the instructed task within the step budget.
- *Relayout.* The target object and goal region are relocated before the first policy observation. A trial is valid once the environment is successfully initialized with the prescribed relocated configuration. All subsequent policy outcomes are included in FCSR, including approaching the original target or goal location, manipulation failures, and premature termination. Success requires completing the instructed task within the step budget.

These criteria exclude failures before the drop trigger for Object Drop and include all validly initialized trials for Distractor Swap and Relayout. All subsequent policy failures remain in the FCSR denominator. For Distractor Swap and Relayout, FCSR therefore coincides with SR over validly initialized trials.

C. Task Definitions and Evaluation Settings

We follow the FCSR definition and the conditional evaluation protocol introduced in the main paper. Below, we provide task-specific evaluation details omitted from the main text. For all tasks in the False Completion Benchmark Suite, an episode is considered successful only when the agent completes the instructed goal within the predefined step budget. An episode is successful only if the instructed goal is achieved within the predefined step budget; premature termination and incorrect placement are counted as failures.

Step Budget. The three perturbation families differ in whether the disturbance introduces additional action requirements. For Distractor Swap and Object Relayout, the perturbation only

alters the initial scene configuration without increasing manipulation complexity. The number of required actions remains comparable to the original LIBERO tasks, and therefore, the maximum episode length is kept unchanged. In contrast, Object Drop requires the agent to detect the unexpected displacement, relocate the fallen object, and perform an additional re-pick action. This introduces extra motion beyond the demonstration regime. To ensure that policies have sufficient budget to perform recovery, we extend the maximum step limit by an additional 30 to 50 steps, depending on the specific drop variant.

Drop Trigger Rule. For Object Drop tasks, we design a deterministic and reproducible drop triggering mechanism to guarantee fair comparisons across models. A drop event is initiated only when the agent maintains a continuous gripper closing state for a fixed duration, indicating that the object has been securely grasped and lifted. Once this condition is satisfied, the environment forces the gripper to open, causing the object to fall from a consistent spatial location. This rule ensures that all models experience the same predefined triggering rule and isolates the evaluation of false completion behavior to perception and decision making rather than random disturbances.

Fairness Across Perturbations. For Distractor Swap and Object Relayout, perturbations are applied deterministically at environment initialization. Since the instruction remains unchanged and the agent starts from the same initial state distribution, all models operate under identical scene configurations. For Object Drop, the deterministic drop trigger further ensures that each model receives an identical disturbance during execution. Together, these settings guarantee that the benchmark provides a controlled and strictly fair evaluation of policies under visually apparent distribution shifts that may induce false completion.

D. LIBERO & RoboTwin 2.0 Benchmark Details

LIBERO Simulation Benchmark. We evaluate ReViP in the LIBERO simulation benchmark [6], a standardized collection of language-conditioned robotic manipulation tasks designed to probe multimodal grounding, visual reasoning, and action generation. Compared with earlier platforms such as RLBench [7], LIBERO provides richer language instructions, more diverse scene layouts, and a broader range of object-level interactions, making it a more suitable testbed for evaluating VLAs.

LIBERO consists of four curated task suites: Spatial, Object, Goal, and Long. Each suite contains ten manipulation tasks accompanied by 50 human teleoperated demonstrations. These suites are designed to decouple different aspects of visuomotor reasoning:

- **LIBERO Spatial** tests spatial reasoning by placing identical objects in varying spatial configurations. Success requires interpreting relations such as left, right, in front of, or behind as described in the instruction.
- **LIBERO Object** evaluates generalization across object categories. While spatial layouts are fixed, the target and distractor objects vary in type, shape, and color, requiring accurate object-level grounding from both vision and language.
- **LIBERO Goal** probes goal-oriented understanding by varying the goal specification while holding object identities and spatial layouts constant. Fine-grained semantic distinctions in instructions must be mapped to distinct manipulation outcomes.
- **LIBERO Long** introduces long-horizon tasks involving multiple intermediate steps, diverse objects, and broader spatial compositions. Successful execution requires perception grounding, semantic tracking, and sequential planning.

ReViP is trained and evaluated under the same standard LIBERO protocol as OpenVLA [8] and π_0 to ensure comparability. By covering spatial reasoning, object identification, goal disambiguation, and long-horizon sequencing, the LIBERO benchmark offers a comprehensive and controlled environment for assessing the robustness and visual grounding capabilities of vision-language action models. This makes LIBERO an appropriate platform for evaluating the contributions of task stage modulation in ReViP.

LIBERO-Plus Benchmark. We further evaluate ReViP on LIBERO-Plus [9], a robustness-oriented extension of LIBERO that introduces controlled variations to the original manipulation environments. Following the evaluation setting in the main paper, we consider four representative perturbation dimensions that directly modify task-relevant evidence: *Background*, *Language*, *Layout*, and *Light*. These variations respectively alter scene appearance, linguistic expression, spatial configuration, and illumination while preserving the underlying manipulation objective.

Unlike our False-Completion Benchmark, LIBERO-Plus does not explicitly construct visual-progress conflicts or require a designated conflict trigger. It instead evaluates whether a policy remains robust when the visual, semantic, or spatial evidence associated with a familiar task changes. We follow the same evaluation protocol for all compared methods and report average task success rate (SR) under each perturbation dimension. This provides a complementary measure of ReViP's generalization beyond explicit false-completion scenarios.

RoboTwin 2.0 Benchmark. RoboTwin 2.0 [10] is a scalable simulation framework and benchmark for robust bimanual robotic manipulation. It instantiates 50 dual-arm tasks across five robot embodiments and provides a large-scale dataset of clean expert trajectories. At the core of RoboTwin 2.0 is a diverse object library and an automatic expert data synthesis pipeline that uses multimodal language models and simulation-in-the-loop refinement to generate realistic task-level execution code. To enhance sim-to-real transfer and robustness, RoboTwin 2.0 applies structured domain randomization along five axes, including clutter, lighting, background, tabletop height, and language instructions, and defines unified evaluation protocols for dual-arm policies [10].

In our study, RoboTwin 2.0 serves as a complementary benchmark to LIBERO and is used to assess whether ReViP extends effectively from single-arm settings to more challenging dual-arm coordination. We select four representative tasks

from the 50-task suite that cover typical bimanual interaction patterns and object configurations. Following the official RoboTwin 2.0 protocol, we fine-tune each model with 50 clean expert demonstrations per task and evaluate it with 100 rollouts under the Hard setting with domain randomization. This evaluation regime imposes a strong robustness requirement and provides a stringent test of the compared vision language action policies.

E. Real-World Setup

1) Real-World Settings and Data Collection.: Our real-world experiments are performed on a 6-DoF ROKAE robotic arm equipped with a JODELL parallel-jaw gripper. We collect real-world successful demonstration trajectories for the Robot Arm via manually moving the robot along the desired path. At each control step, the policy receives a main image and wrist image and a 7-dimensional proprioceptive state in joint space (6 joint positions and 1 gripper scalar). The model predicts a chunk of 50 actions, each specifying six joint-position commands and one gripper command.

For each task, we collect 50-100 demonstrations by kinesthetic teaching through the desired manipulation trajectory. All compared methods are trained using the same demonstrations and evaluated under identical robot, camera, and task configurations.

2) Real-World Tasks and Evaluation Protocols: We evaluate ReViP under two complementary real-world settings: a *Controlled FC Setting* with explicitly induced visual-progress conflicts, and a *Challenging Real-World Setting* with broader sensing and scene variations. The evaluated tasks are summarized below.

Controlled FC Setting.

- *"Pick up the cup on the white plate, then place it inside the white plastic square box."*
  This task evaluates standard object placement with a stable visual reference. The white plate provides a distinctive anchor for target localization. We additionally apply controlled *Object Drop* perturbations after a valid grasp to evaluate whether the policy can revise its presumed task progress and recover.
- *"Pick up the toy and place it in the open drawer, then close the drawer."*
  This task evaluates multi-stage manipulation involving grasping, placement, and drawer closing. It requires the policy to maintain correct task-stage progression across a longer execution horizon and to recover when object possession is unexpectedly invalidated.

Challenging Real-World Setting.

- *"Pick up the cup on the table, then place it inside the white plastic square box."*
  This variant removes the plate anchor and introduces a high-contrast green tabletop, increasing the difficulty of target localization under changed scene appearance. Additional Drop and Distractor conditions are used to evaluate robustness under combined visual and execution disturbances.
- *"Pick up the small red cube on the table, then place it inside the white box."*
  This task evaluates fine-grained localization and grasping of a small, low-profile target. Distractor objects are introduced around the workspace to increase ambiguity and test whether the policy maintains grounding to the instructed target.
- *"Pick up the purple toy on the table, then place it into the middle drawer of the cabinet, and finally close the drawer."*
  This long-horizon task combines grasping, precise placement, and drawer closing in a cluttered scene containing multiple objects with diverse appearances. It evaluates visual grounding, task-stage tracking, and corrective action generation under increased scene complexity and execution horizon.

Across all real-world settings, ReViP maintains substantially higher robustness under perturbations and complex visual clutter.

Drop Trigger Mechanism We evaluate the model's response to false-completion scenarios using a controlled Drop perturbation. To ensure **repeatability, fairness, and naturalistic failure behavior**, we implement Drop using a hardware-in-the-loop condition: First, during execution, we continuously monitor the gripper width in real time. Once the gripper closes beyond a threshold, indicating that it has successfully grasped the object, the robot proceeds with the placement trajectory. Then, at a predefined early stage of this motion, we command the gripper to open, causing the object to fall naturally along its original motion path.

Distractor Setting. For Distractor conditions, visually competing objects are introduced around the target while the task instruction remains unchanged. The target identity is preserved, requiring the policy to maintain grounding to the linguistically specified object under increased visual ambiguity.

Scene and Sensing Variations. The Challenging Real-World Setting additionally varies the background appearance, camera configuration, target scale, object clutter, and task horizon. Selected conditions use an ORBBEC DaBai DCW camera and a high-contrast green tabletop background, while cluttered scenes contain multiple objects with diverse shapes and appearances. These variations alter the visual evidence available to the policy without changing the underlying task objective.

Evaluation Protocol. All methods use the same task demonstrations and evaluation conditions, with 10 real-world trials conducted for each condition. The Controlled FC Setting evaluates recovery after explicitly induced physical visual-progress conflicts, whereas the Challenging Real-World Setting reports end-to-end task success under broader real-world variations.

IV. Supplementary Quantitative Analysis

This section extends the main evaluations with additional results on bimanual manipulation, sensitivity to modulation and distillation settings, and robustness to observer errors. These experiments examine the applicability of ReViP and its sensitivity to design choices and imperfect task-stage cues.

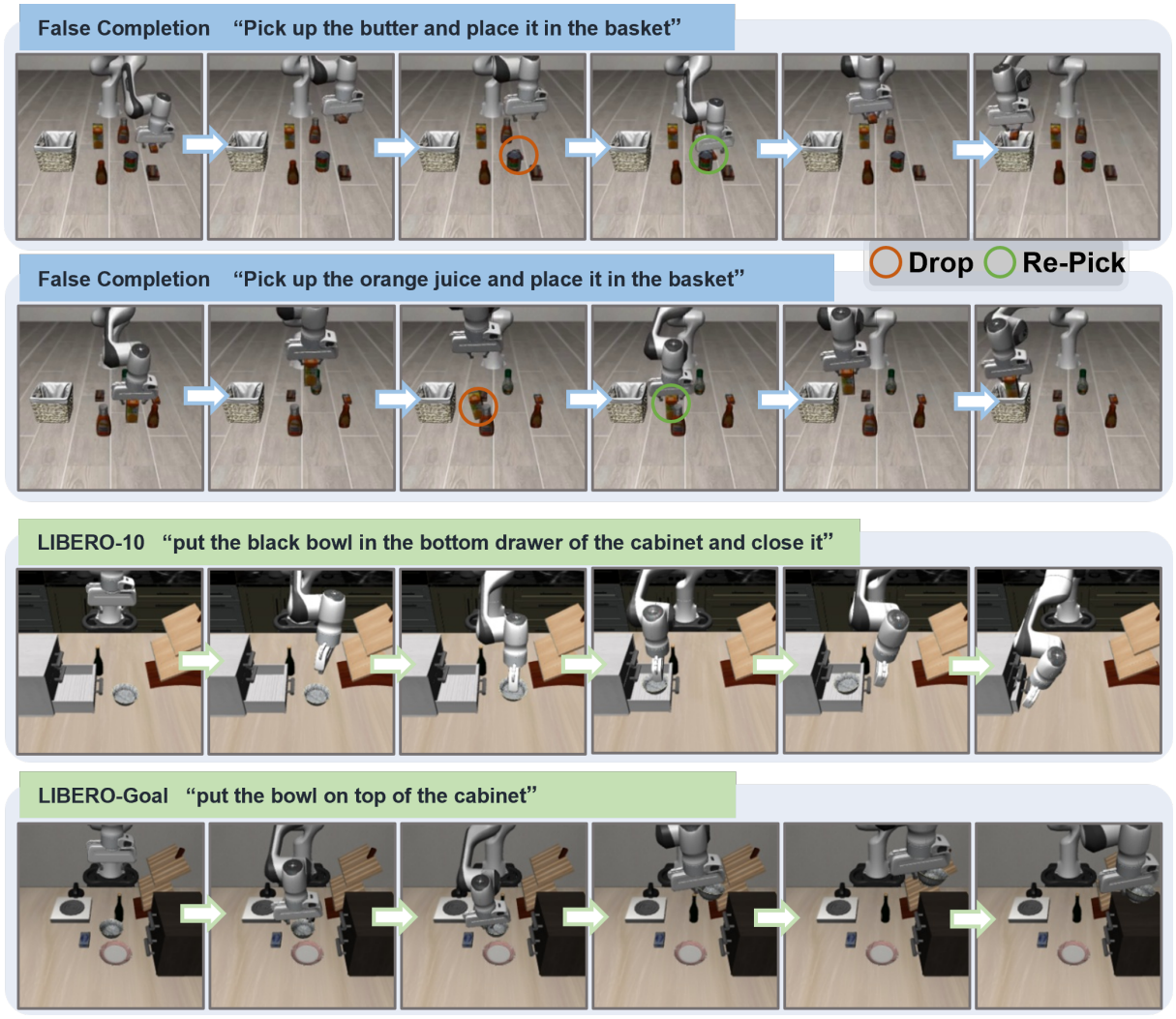


Fig. 2. Additional qualitative results in simulation on the False-Completion Benchmark, LIBERO-10, and LIBERO-Goal. In the False-Completion Benchmark, ReViP detects the object drop during execution and re-grasps the target, achieving true completion instead of prematurely terminating. On LIBERO-10 and LIBERO-Goal, ReViP consistently completes long-horizon tasks under diverse disturbances, environments, and object configurations. These visualizations highlight the complexity of the tasks and further demonstrate ReViP's robustness, generalization, and execution capabilities.

TABLE III
BIMANUAL MANIPULATION ON ROBOTWIN 2.0 UNDER THE OFFICIAL EASY-TO-HARD PROTOCOL. THE BEST RESULTS ARE SHOWN IN BOLD.

Task	DP3 [11]	RDT [12]	π_0 [2]	ReViP
Place Object Stand	0%	5%	11%	**22%** (+11%)
Place Bread Basket	1%	2%	4%	**13%** (+9%)
Pick Dual Bottles	1%	13%	12%	**21%** (+9%)
Put Bottles Dustbin	21%	4%	13%	**24%** (+11%)
Average	6%	6%	10%	**20%** (+10%)

TABLE IV
ABLATION STUDY: FACTORS α. AVERAGE FALSE COMPLETION SUCCESS RATES (FCSR) ON THE FALSE-COMPLETION BENCHMARK ARE REPORTED.

Factors	0	0.25	0.5	1.0	ReViP
α in FiLM	39%	56%	61%	46%	**63%**

A. Generalization to Bimanual RoboTwin 2.0.

To evaluate whether ReViP generalizes beyond single-arm manipulation, we further test it on four representative contact-rich tasks from RoboTwin 2.0 [10]. Each task is trained with 50 demonstrations collected in clean environments and evaluated over 100 trials under *hard-mode* conditions with domain randomization, lighting variations, object clutter, and table-height shifts. As shown in Table III, ReViP achieves the best performance, improving its π_0 backbone from 10% to 20% while consistently outperforming DP3 and RDT. These results demonstrate that ReViP scales effectively to coordinated bimanual manipulation under challenging environmental variations.

B. Sensitivity to Modulation Strength

We further analyze the sensitivity of ReViP to the modulation strength α in FiLM. We replace the learned values with fixed settings while keeping all other components unchanged. As shown in Table IV, moderate strengths generally outperform overly aggressive modulation or intervention. For FiLM,

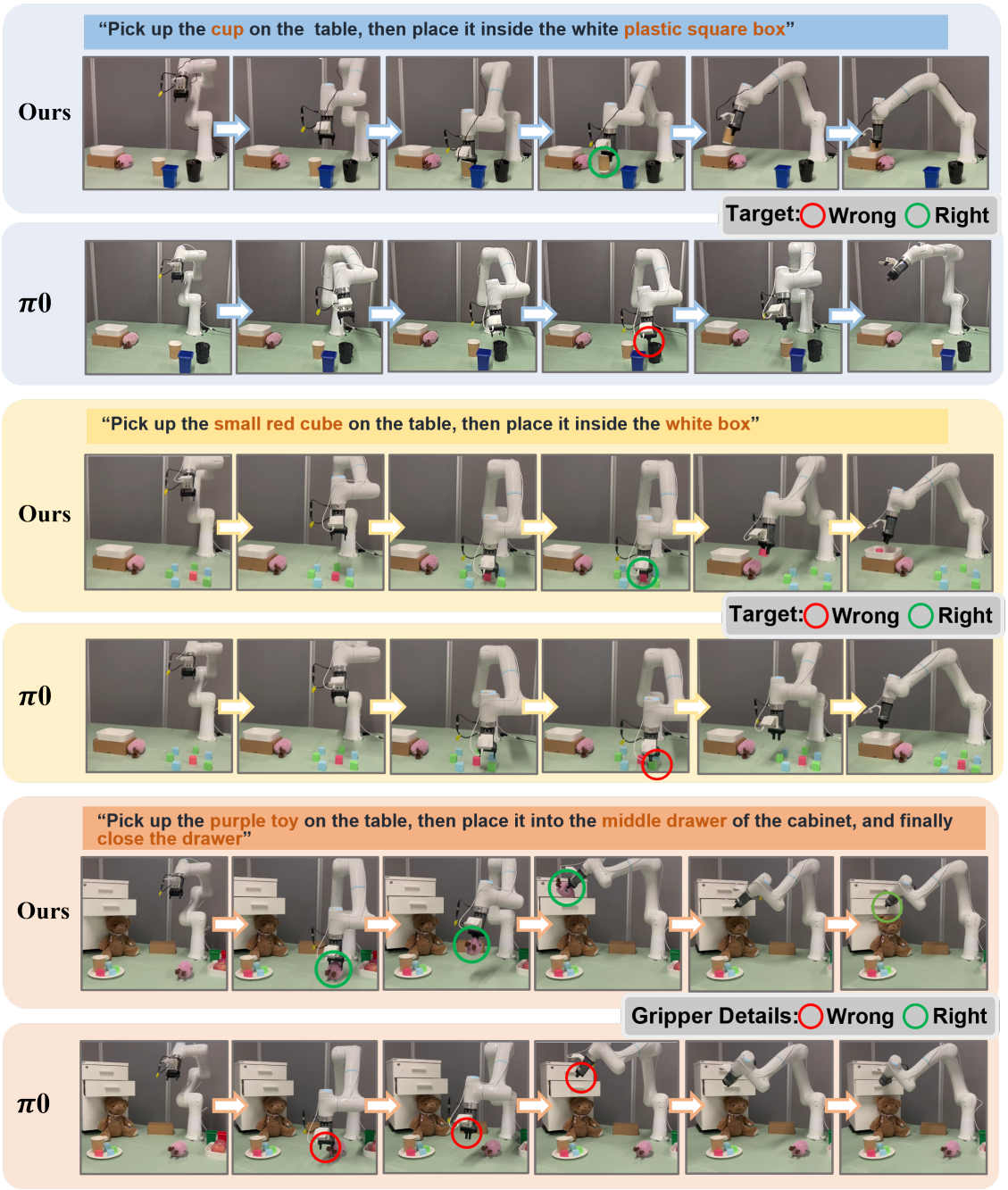


Fig. 3. Additional qualitative real-world comparisons between ReViP and π_0 on challenging false-completion scenarios and long-horizon manipulation tasks. ReViP accurately identifies the correct target in cluttered scenes and avoids mis-grasp-induced false completion, while π_0 is easily confused by distractors. For long-horizon drawer tasks, ReViP re-localizes the target and completes the full sequence, whereas π_0 often proceeds with incorrect state-driven actions.

fixed α remains effective across a range of values, while the learned setting achieves the best overall performance.

C. Sensitivity to Scale-Alignment Weight

We further analyze the sensitivity of ReViP-Flash to the scale-alignment weight λ_{scale} in the main paper. We vary λ_{scale} while retaining both $\mathcal{L}_{\text{sim}}$ and $\mathcal{L}_{\text{action}}$. As shown in Figure 5, FCSR improves from 59% at $\lambda_{\text{scale}} = 0$ to 62% at 0.01, and decreases to 61% and 58% at 0.1 and 1.0, respectively. This result indicates that moderate scale alignment improves the compatibility between distilled cues and the downstream ReViP modules, whereas excessive weighting can over-constrain the cue representation. We therefore set $\lambda_{\text{scale}} = 0.01$ throughout our experiments.

D. Robustness to Observer Errors

To assess ReViP's robustness to observer errors, we examine naturally occurring errors and controlled cue corruption. Across 1,646 observer predictions from 400 rollouts, persistent errors ($\geq$ 3 consecutive erroneous chunks) occur in only 3 rollouts. We then replace the inferred stage or target cue with an incorrect one at each action chunk with probability

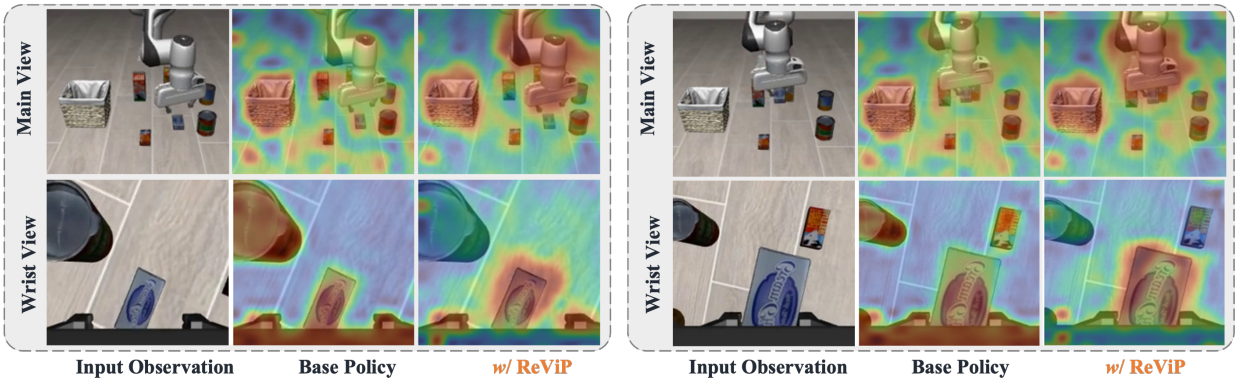


Fig. 4. **Visualization of Visual-progress grounding.** Compared with the base policy, ReViP produces more concentrated visual responses around progress-relevant regions, including the robot embodiment, manipulated objects, and their interaction areas.

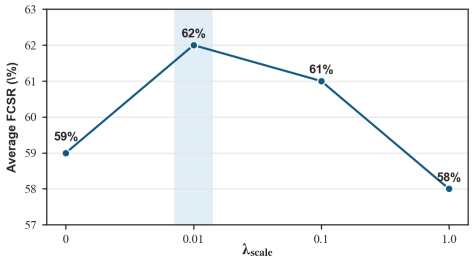


Fig. 5. Sensitivity of ReViP-Flash to λ_{scale}. Average FCSR on the False-Completion Benchmark is reported.

TABLE V
ROBUSTNESS TO CORRUPTED TSO CUES. FCSR AND MEAN SUCCESSFUL-EPISODE STEPS UNDER CONTROLLED WRONG-STAGE AND WRONG-TARGET CUE CORRUPTION. WE REPORTED THE RELATIVE INCREASE IN SUCCESSFUL-EPISODE STEPS OVER THE CLEAN SETTING.

Corrupt. p	Wrong Stage		Wrong Target	
	FCSR ↑	Succ. Steps ↓	FCSR ↑	Succ. Steps ↓
Clean	63.0%	192.2	63.0%	192.2
5%	60.2%	200.2 (+4.2%)	60.8%	198.7 (+3.4%)
10%	56.8%	211.8 (+10.2%)	57.5%	209.3 (+8.9%)
15%	54.9%	232.3 (+20.9%)	55.4%	225.4 (+17.3%)

p. As shown in Table V, performance degrades gradually as corruption increases. At $p = 15\%$, ReViP retains 54.9% and 55.4% FCSR under wrong-stage and wrong-target corruption, respectively, exceeding the 36.0% π_0 baseline. The corresponding successful rollouts require 20.9% and 17.3% more execution steps, respectively, reflecting that occasional cue errors can be accommodated through subsequent closed-loop correction.

Overall, these results support ReViP's applicability across VLA backbones and observer models, as well as its robustness to imperfect visual-progress cues.

V. SUPPLEMENTARY QUALITATIVE ANALYSIS

This section presents additional qualitative results from simulation and real-world experiments, focusing on challenging false-completion scenarios and broader manipulation settings. We also visualize task-relevant visual responses to complement the behavioral comparisons.

A. Additional Visualizations of Simulation Results

As shown in Figure 2, we provide additional simulation results on the False-Completion Benchmark and LIBERO. Across diverse perturbations, environments, and object configurations, ReViP consistently revises its behavior according to current visual evidence and completes multi-stage manipulation tasks. These examples further illustrate its robustness and generalization beyond the quantitative results reported in the main paper.

B. Additional Visualizations of Real-World Results

Figure 3 provides qualitative comparisons between ReViP and the baseline π_0 across representative false-completion and long-horizon real-world scenarios. These examples highlight whether task-relevant visual evidence changes subsequent execution, particularly when the observed scene no longer supports the policy's presumed task progress.

In cluttered scenes with visually similar distractors, ReViP maintains grounding to the instructed target, distinguishing the intended cup or red cube from competing objects. By comparison, π_0 may select an incorrect object and continue with the subsequent transport and placement motions. The relevant failure extends beyond incorrect target selection: execution advances as if the target-acquisition stage had been completed, despite the mismatch between the manipulated object and the instruction. This illustrates how a locally plausible manipulation sequence can proceed under an unsupported progress assumption.

For long-horizon manipulation, π_0 may continue to later task stages even when object possession or placement has been visually invalidated, such as closing the drawer after an unsuccessful grasp. ReViP instead revises execution to address the unmet condition, re-localizing and grasping the target before completing placement and drawer closing. The distinction lies in whether current visual evidence redirects the remaining action sequence.

These behaviors are consistent with the two complementary mechanisms of ReViP. Visual-Progress Grounding supplies task-stage evidence that conditions policy representations, while Stage-Aware Attention Rebalancing adjusts the influence

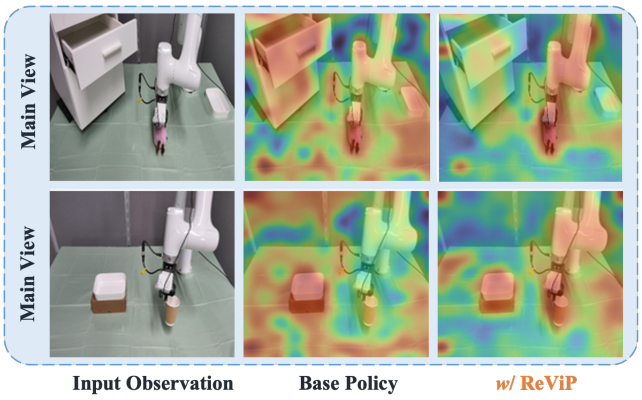


Fig. 6. **Visualization of Visual-progress grounding.** Compared with the base policy, ReViP produces more concentrated visual responses around progress-relevant regions, including the robot embodiment, manipulated objects, and their interaction areas.

of visual and execution cues during action generation. As observations are updated, this pathway allows corrective actions to be incorporated into ongoing execution. The visualizations thus provide behavioral evidence of improved action revision under visual-progress conflicts, complementing the quantitative evaluations and modality-controlled diagnostics.

C. Visualization Analysis of Visual-Progress Grounding

To further understand how ReViP changes the evidence used for action generation, we provide additional visualization analysis of its visual-progress rebalancing mechanism. Unlike the qualitative rollouts above, which visualize behavioral outcomes, this analysis examines where task-stage-relevant visual evidence is localized and how its contribution to action generation changes under visual-progress conflicts.

We visualize task-conditioned visual responses extracted from the vision encoder (*i.e*, SigLIP [13]) of the base π_0 and ReViP checkpoints. Patch-level responses are projected back to the image plane, and their difference is used to highlight regions whose visual relevance is strengthened or suppressed by ReViP. For comparisons between successful and failed rollouts, frames are aligned according to semantic task stages rather than absolute timesteps.

We further provide additional visualization results in both simulation and the real world to examine how ReViP changes the task-relevant visual responses used for action generation.

Simulation analysis. As shown in Figure 4, under representative false-completion scenarios, the base policy exhibits relatively diffuse visual responses in both the main view and the wrist view, including responses over background regions and distractor objects. In contrast, ReViP produces substantially more concentrated responses around task stage-relevant regions, particularly the manipulated object, the robot embodiment, and their interaction area. This effect is consistently observed across the two complementary viewpoints. The main view helps localize the target object within the global scene, while the wrist view emphasizes local robot–object interaction cues that are directly relevant to grasping and contact. Such cross-view concentration suggests that visual-progress grounding enables the policy to suppress irrelevant scene distractions and assign greater weight to the visual evidence that is most informative for the current task stage.

Real-world analysis. A similar pattern is observed in the real-world visualization results in Figure 6. Compared with the base policy, ReViP exhibits more focused responses around the target object and robot–object interaction regions despite appearance variation and clutter. This result is particularly important because real-world observations contain richer appearance variation, sensing noise, and clutter than simulations. The persistence of the same response pattern therefore supports that the visual-progress rebalancing mechanism learned by ReViP transfers beyond simulation and remains effective in practical robotic manipulation settings.

Overall, these additional visualizations are consistent with the intended role of visual-progress grounding and stage-aware attention rebalancing. ReViP exhibits more concentrated visual responses around progress-relevant regions that support the current task stage. Combined with the rollout results, these observations suggest that ReViP better translates updated visual evidence into corrective actions, helping close the perception-action loop under visual-progress conflicts.

REFERENCES

[1] S. Bai, K. Chen, X. Liu, J. Wang, W. Ge, S. Song, K. Dang, P. Wang, S. Wang, J. Tang, H. Zhong, Y. Zhu, M. Yang, Z. Li, J. Wan, P. Wang, W. Ding, Z. Fu, Y. Xu, J. Ye, X. Zhang, T. Xie, Z. Cheng, H. Zhang, Z. Yang, H. Xu, and J. Lin, "Qwen2.5-vl technical report," *arXiv preprint arXiv:2502.13923*, 2025.

[2] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, "π_0: A vision-language-action flow model for general robot control," *Robotics: Science and Systems*, 2025.

[3] S. Chen, H. Hadfield, A. Zook, M. A. Uy, C. H. Song, E. Coumans, X. Yang, F. Ladhak, Q. Qu, S. Birchfield *et al.*, "Volo: A physical orchestrator for open-vocabulary long-horizon manipulation," *arXiv preprint arXiv:2606.07723*, 2026.

[4] Y. Zhang, H. Zhang, F. Gao, X. Li, Z. Liu, C. Zhu, J. Qiu, Y. Yan, J. Liu, W. Tang *et al.*, "Harness vla: Steering frozen vlas into reliable manipulation primitives via memory-guided agents," *arXiv preprint arXiv:2607.08448*, 2026.

[5] X.-Y. Zhou, Y. Xu, G. Tie, Y.-C. Chen, G.-W. Zhang, D.-F. Chu, P. Zhou, and L. Sun, "Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization," *ArXiv*, vol. abs/2510.03827, 2025. [Online]. Available: https://api.semanticscholar.org/CorpusID:281843691

[6] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "Libero: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems*, 2023.

[7] S. James, Z. Ma, D. Rovick Arrojo, and A. J. Davison, "Rlbench: The robot learning benchmark & learning environment," *IEEE Robotics and Automation Letters*, 2020.

[8] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "Openvla: An open-source vision-language-action model," in *Conference on Robot Learning*, 2024.

[9] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei *et al.*, "Libero-plus: In-depth robustness analysis of vision-language-action models," *arXiv preprint arXiv:2510.13626*, 2025.

[10] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu *et al.*, "Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation," *arXiv preprint arXiv:2506.18088*, 2025.

[11] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, "3d diffusion policy," *arXiv preprint arXiv:2403.03954*, 2024.
[12] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, "RDT-1b: a diffusion foundation model for bimanual manipulation," in *The Thirteenth International Conference on Learning Representations*, 2025.
[13] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid loss for language image pre-training," *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 11 941–11 952, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:257767223